PhishVote: An Adaptive Soft-Voting Ensemble of Tree-Based Classifiers for Phishing Website Detection

Zander R. Duhaylungsod
Bachelor in Computer Science
Matina, Davao City, 8000
z.duhaylungsod.547209@umindanao.edu.ph

Annika Dumalogdog
Bachelor in Computer Science
Matina, Davao City, 8000
a.dumalogdog.547692@umindanao.edu.ph

Categories and Subject Descriptors

Computing Methodologies → Machine learning → Algorithms → Ensemble Methods

General Terms

Algorithms, Performance, Reliability, Experimentation, Security, Standardization.

Keywords

Phishing Detection; Machine Learning; Ensemble Learning; Weighted Soft Voting; Gradient-Boosted Trees; Tree-Based Classifiers; Cybersecurity

Abstract

Phishing websites remain a persistent and damaging form of cybercrime, necessitating robust automated detection mechanisms that can generalize across evolving attack patterns. This study proposes PhishVote, an adaptive soft-voting ensemble classifier for phishing website detection that integrates five tree-based models (Random Forest, XGBoost, CatBoost, LightGBM, and Gradient Boosting) through a rank-based, cross-validated AUC weighting scheme. PhishVote is evaluated on the UCI Phishing Websites dataset (11,054 records, 30 ternary-encoded features) against the published results of Saeed [1], a hard voting ensemble of Logistic Regression, Gradient Boosting, and K-Nearest Neighbors that reported 95.02% accuracy on the same dataset. The preprocessing pipeline enforces stratified 80/20 partitioning and conditional SMOTE oversampling applied exclusively to the training split to ensure methodologically sound evaluation. Experimental results show that PhishVote achieves 97.42% accuracy, 0.97 precision, 0.98 recall, and an F1-score of 0.98 on the held-out test set, surpassing the Saeed [1] baseline across all four evaluation metrics by 2.40 percentage points in accuracy and 0.02 in F1-score. Every individual PhishVote base learner outperformed the Saeed [1] Hard Voting Ensemble, with even the weakest base model matching the baseline's F1-score of 0.96, indicating that the upgrade to high-capacity tree-based architectures is the primary driver of performance improvement. These results confirm the research hypothesis and establish PhishVote as a methodologically sound, reproducible ensemble framework for phishing website classification.

1. INTRODUCTION

1.1 Background of the Study

Phishing attacks remain one of the most pervasive and damaging forms of cybercrime in the modern digital landscape. According to the Anti-Phishing Working Group (APWG) [2], phishing incidents have grown dramatically year over year, with millions of unique phishing URLs detected annually, targeting financial institutions, e-commerce platforms, and individual users worldwide. A phishing website is a fraudulent web page designed to impersonate a legitimate service, deceiving users into surrendering sensitive credentials such as passwords, banking information, and personal identification numbers. The consequences range from individual financial loss to large-scale corporate data breaches, making robust, automated detection mechanisms an urgent priority for cybersecurity practitioners and researchers alike.

According to Sahoo et al. [3], Machine learning (ML) has emerged as a powerful paradigm for phishing detection, offering the ability to learn discriminative patterns from collections of website features without relying solely on rule-based blacklists that are inherently reactive and easily circumvented. Additionally, Dietterich [4] discusses how the Feature-based ML classifiers can evaluate structural and behavioral website attributes (such as URL characteristics, domain properties, and page-level indicators) to distinguish phishing pages from legitimate ones in real time. Ensemble methods, in particular, have demonstrated superior classification performance by combining the predictions of multiple base learners, thereby reducing variance and improving generalization.

A prominent recent contribution in this area is the work of Saeed [1], who proposed an ensemble voting classifier combining Logistic Regression, Gradient Boosting, and K-Nearest Neighbors (KNN) on the UCI Phishing Websites dataset. Saeed [1] reported accuracy figures of up to 95.02% using hard voting aggregation, establishing a widely cited benchmark for ensemble-based phishing detection using classical ML models. The study demonstrated that combining heterogeneous classifiers through a voting scheme could yield measurable improvements over individual learners, providing a strong foundation for further research in ensemble-based phishing detection. The UCI Phishing Websites dataset, originally compiled by Mohammad et al. [5], contains 11,054 records with 30 heuristic features encoded as ternary values (-1, 0, 1), representing various URL and webpage characteristics.

Related works in phishing detection affirm the viability of ML ensemble approaches using website-centric feature sets. Muhammad et al. [6] established that gradient boosting and tree ensemble methods consistently outperform linear classifiers on phishing feature sets, while Vrbančič et al. [7] validated tree-based models including Random Forest and XGBoost on URL-based classification tasks. Abdelhamid et al. [8] demonstrated that information-gain-ranked feature selection preserved near-full accuracy with reduced dimensionality, and Rao and Pais [9] showed that hybrid URL and content-based feature sets achieve competitive phishing detection performance.

This study proposes PhishVote, a novel adaptive soft-voting ensemble classifier for phishing website detection that builds upon the foundation established by Saeed [1] while introducing several methodological enhancements. PhishVote integrates five high-capacity tree-based boosting models—Random Forest, XGBoost, CatBoost, LightGBM, and Gradient Boosting—through an adaptive soft-voting aggregation scheme where models are ranked by cross-validated AUC scores and their predictions are weighted accordingly. Unlike the hard voting approach used by Saeed [1], which treats all base classifiers equally, PhishVote's adaptive weighting mechanism ensures that more consistently accurate models contribute proportionally more to final predictions. The study evaluates PhishVote against the Saeed [1] baseline on the UCI Phishing Websites dataset using standardized preprocessing protocols including stratified 80/20 partitioning and feature scaling, ensuring fair and reproducible comparison. By maintaining identical dataset conditions, this research isolates the performance gains attributable to the enhanced ensemble architecture and adaptive voting mechanism.

Table 1. Related Studies

Ref No.	Language	Approach	Results	Gaps Addressed
[1]	English	Hard Voting Ensemble of Logistic Regression, Gradient Boosting, and KNN	Achieved 95.02% accuracy on the UCI Phishing Websites dataset using hard voting aggregation.	No class imbalance handling, non-stratified splitting, and uniform voting weights.
[6]	English	Comparative study of ML models including tree-based boosting classifiers	Gradient boosting and tree ensembles consistently outperformed linear classifiers on phishing website feature sets.	No ensemble voting aggregation, no oversampling for class imbalance, and no adaptive model weighting.
[7]	English	Benchmarking of Random Forest and XGBoost on URL-based phishing datasets	Validated effectiveness of tree-based models on URL and webpage feature classification tasks.	No adaptive soft-voting aggregation, no class imbalance handling, and no cross-validated model weighting.
[8]	English	Associative classification with information-gain-ranked feature selection for phishing detection	Near-full classification accuracy was preserved using a reduced feature subset.	No ensemble voting mechanism, no class imbalance handling, and no adaptive weighting of model contributions.
[9]	English	Systematic analysis of URL and content-based hybrid feature sets for phishing detection	Hybrid feature sets achieved competitive classification performance, validating complementary value.	No ensemble aggregation strategy, no oversampling for class imbalance, and no cross-validated model weighting.

As illustrated in Table 1, none of the reviewed studies simultaneously address class imbalance, stratified data partitioning, and adaptive ensemble weighting, the three methodological gaps that PhishVote is designed to resolve. While Saeed [1], Muhammad et al. [6], and Vrbančič et al. [8] each contribute ensemble or tree-based approaches to phishing detection, none apply SMOTE for imbalance handling, none enforce stratified train-test splits, and none weight base classifiers by cross-validated performance. PhishVote addresses all three gaps through conditional SMOTE oversampling, stratified 80/20 partitioning, and rank-based CV-AUC soft voting across five tree-based classifiers.

1.2 Purpose and Description

Phishing attacks remain one of the most persistent vectors of cybercrime, exploiting human trust to compromise credentials, financial assets, and sensitive organizational data. Despite decades of research, real-time phishing detection remains an open challenge as adversaries continuously adapt to evade blacklists and signature-based filters. This study addresses this gap by developing PhishVote, an adaptive soft-voting ensemble classifier that tackles methodological weaknesses in prior work (namely unaddressed class imbalance, non-stratified data partitioning, and the absence of adaptive weighting mechanisms) as observed in Saeed [1]. The timeliness of this research is further underscored by the continued global rise in phishing incidents and the growing need for detection methodologies that leverage the full potential of ensemble learning.

PhishVote contributes to the machine learning and cybersecurity literature by establishing a reproducible ensemble framework for phishing website classification on the UCI Phishing Websites dataset. The study aligns with UN Sustainable Development Goals 9 and 16, supporting safe and resilient digital infrastructure, and is anchored in the College of Computing Education's research agenda on intelligent systems and data-driven security solutions. Direct beneficiaries include cybersecurity researchers, internet service providers, browser developers, and end users across academic, governmental, and commercial institutions.

1.3 Objectives

1.1.1 General Objective

This study aims to design, develop, and evaluate PhishVote, an adaptive soft-voting ensemble classifier for phishing website detection, using tree-based models and preprocessing protocols on the UCI Phishing Websites dataset to produce a methodologically sound phishing detection system that advances ensemble-based classification approaches.

1.1.2 Specific Objectives

1.1.2.1 To design a phishing detection pipeline integrating StandardScaler normalization and stratified 80/20 splitting to ensure proper feature scaling and equitable class representation.

1.1.2.2 To develop PhishVote, an adaptive soft-voting ensemble of Random Forest, XGBoost, CatBoost, LightGBM, and Gradient Boosting with rank-based cross-validated AUC weighting for binary phishing classification.

1.1.2.3 To evaluate PhishVote against the Saeed [1] baseline in terms of accuracy, precision, recall, and F1-score on the UCI Phishing Websites dataset.

1.1.2.4 To benchmark for comparative evaluation in phishing website detection.

1.4 Scope and Limitation

This study covers the design, development, and evaluation of PhishVote, a machine learning-based ensemble classifier for phishing website detection, conducted during the academic year 2024-2025 at the University of Mindanao, Matina, Davao City. The scope is confined to feature-based binary classification using the UCI Phishing Websites dataset (11,054 records, 30 ternary-encoded heuristic features). The system operates in an offline, batch-evaluation context intended for cybersecurity researchers and academic practitioners. Users supply a preprocessed feature matrix through a Python-based pipeline, and the system returns binary classification labels alongside soft-vote confidence scores. The study includes a comparative evaluation of PhishVote against the published Saeed [1] baseline across standard classification metrics.

The study is subject to several limitations. First, PhishVote is trained on a static dataset and does not incorporate real-time URL processing or online learning, meaning deployment performance on live phishing campaigns may differ from reported metrics. Second, the system relies solely on the 30 UCI heuristic features and does not analyze raw webpage content, DOM structure, SSL certificate metadata, or WHOIS data used in more comprehensive detection frameworks. Third, the UCI dataset was collected in 2015, and modern phishing techniques may not be fully represented. Fourth, while the dataset is relatively balanced, generalizability to datasets with different feature distributions or imbalance ratios remains to be validated.

2. METHODS

This section describes the technical methodology underlying PhishVote, an adaptive soft-voting ensemble classifier for phishing website detection. The methodology is organized to follow the pipeline from data collection through preprocessing, baseline comparison, proposed model design, and evaluation. Key elements include standard scaling for feature normalization, stratified 80/20 train-test partitioning to preserve class ratios, adaptive rank-based soft voting across five tree models, and comparative evaluation against the published results of the Saeed [1] baseline ensemble on the UCI Phishing Websites dataset.

2.1 Conceptual Framework

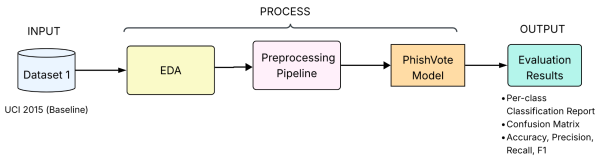

Figure 1. *PhishVote: Proposed Approach*

Figure 1 presents the conceptual framework of PhishVote, illustrating the methodological flow from data input through preprocessing, model development, and evaluation.

The framework begins with a single input: the UCI Phishing Websites dataset [5], containing 11,054 records described by 30 heuristic ternary-encoded features. The dataset undergoes exploratory data analysis followed by a preprocessing pipeline comprising StandardScaler normalization and stratified 80/20 train-test partitioning. The preprocessed data is then passed to the PhishVote Model, an adaptive soft-voting ensemble of five tree-based classifiers: Random Forest, XGBoost, CatBoost, LightGBM, and Gradient Boosting, whose contributions are weighted by rank-based 5-fold cross-validated AUC scores. The framework converges at the Evaluation stage, where PhishVote's classification performance is assessed through per-class classification reports, confusion matrices, and a summary comparison of accuracy, precision, recall, and F1-score against the published Saeed [1] baseline results.

2.2 Data Gathering

A single dataset was used in this study: the UCI Phishing Websites Dataset [5], referred to as DS-Baseline. It was selected because it is the same dataset used by Saeed [1], enabling direct comparison against their published results. DS-Baseline encodes 30 URL and page-level attributes as heuristic ternary values (-1, 0, 1) across 11,054 records, with class labels remapped to 1 (phishing) and 0 (legitimate) for consistency in model training and evaluation.

2.2.1 Data Source

The dataset was obtained from the UCI Machine Learning Repository, a publicly accessible repository of datasets maintained for machine learning research. The specific dataset used is the Phishing Websites Dataset [5], available at https://archive.ics.uci.edu/dataset/327/phishing+websites, last accessed January 2025. The dataset was originally compiled by Mohammad, Thabtah, and McCluskey and has been widely adopted as a benchmark in phishing detection research, including the baseline Saeed [1] study.

2.3 Exploratory Data Analysis

An exploratory data analysis (EDA) was conducted on the dataset to characterize their structure, assess data quality, and inform preprocessing decisions. The EDA examined five aspects: class distribution, missing value audit, feature distribution, correlation heatmap, and target correlation rank.

2.4 Data Preprocessing

A preprocessing pipeline was applied to the dataset before model training. The pipeline was designed to ensure equitable feature contribution, proper data partitioning, and prevention of data leakage between training and test sets. Table 2 describes each step in the order of application.

Table 2. PhishVote Preprocessing Pipeline

#	Step	Description
1	Label Standardization	Class labels are remapped to a unified binary scheme: 1 = phishing, 0 = legitimate.
2	Missing Value Imputation	Any missing feature values are inspected and handled appropriately. The dataset contains no missing values.
3	Stratified 80/20 Split	The dataset is partitioned into 80% training and 20% test subsets using stratified sampling (stratify=y, random_state=42).
4	Feature Scaling	StandardScaler is fit on the training partition only and then applied to the test partition, normalizing all features.
5	Conditional SMOTE	If the minority-to-majority class ratio in the training set is below 0.80, SMOTE is applied to the training partition only [10].

The sequence of these steps is deliberate. Stratified splitting must occur before scaling to prevent test statistics from influencing normalization, and scaling must be fit on the training partition only to ensure that the evaluation reflects genuine out-of-sample generalization. These ordering constraints ensure methodological soundness and fair comparison with the published Saeed [1] baseline results.

2.5 PhishVote

The proposed model PhishVote is an adaptive soft-voting ensemble classifier that builds upon the Saeed [1] baseline through two principal innovations: upgraded base learners and an adaptive weighting scheme. Figure 2 shows the architectural framework of the proposed system, while Figure 3 illustrates the baseline approach for comparison.

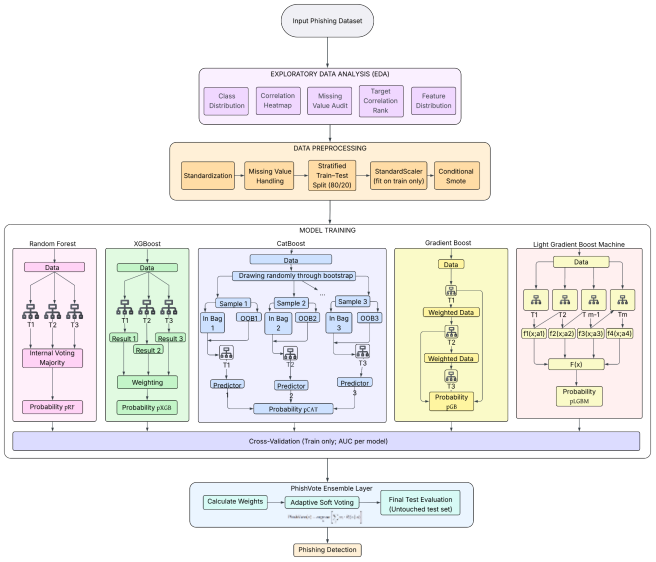

Figure 2. PhishVote Architectural Framework

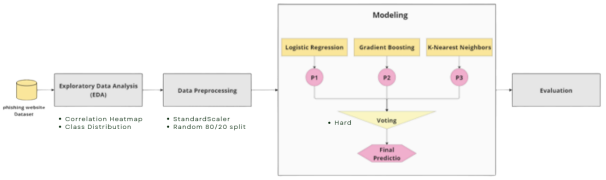

Figure 3. Baseline Model's Architectural Framework

2.5.1 Base Learner Selection

Five tree-based classifiers were selected as the base models of the PhishVote ensemble. These models were chosen for their established superiority on tabular classification tasks and for the diversity of their internal learning mechanisms, which promotes complementary error patterns—a key condition for effective ensemble aggregation [11], [12], [13], [14], [15]. Table 3 lists each model with its library and configuration.

Table 3. PhishVote Base Model Configurations

Model	Library	Configuration
Random Forest	scikit-learn	n_estimators=200, random_state=42, n_jobs=−1
XGBoost	xgboost	use_label_encoder=False, eval_metric="logloss", random_state=42
CatBoost	catboost	verbose=0, random_state=42
LightGBM	lightgbm	random_state=42, verbose=−1
Gradient Boosting	scikit-learn	n_estimators=100, random_state=42
PhishVote (Soft Voting)	—	**Adaptive soft voting; weights by rank of 5-fold stratified CV-AUC; final label = argmax of weighted probability sum**

2.5.1.1 Random Forest

Random Forest (RF) is a bagging ensemble of decision trees, where each tree is trained on a bootstrap sample of the training data and a random subset of features at each split [11]. The final prediction aggregates votes across all trees to reduce variance without increasing bias. Imported from scikit-learn, RF was instantiated with n_estimators=200, random_state=42 for reproducibility, and n_jobs=-1 to enable full parallel training across all available CPU cores. The bootstrap aggregation mechanism further ensures that each tree is exposed to a different subset of training instances.

$$\hat{y} = \arg\max_{c} \sum_{t=1}^{T} \mathbf{1}\left[h_t(x) = c\right]$$

where $h_t(x)$ is the prediction of the t-th decision tree and T=200 is the total number of trees.

2.5.1.2 XGBoost (XGB)

XGBoost (XGB) implements gradient boosting where trees are added sequentially with each learner fitting the residual errors of the ensemble so far, extended with L1 and L2 regularization terms in the objective function, column subsampling, and a sparsity-aware split-finding algorithm that improves both speed and generalization[12]. Imported from the xgboost library, it was configured with use_label_encoder=False to suppress deprecation warnings, eval_metric='logloss' as the evaluation criterion, and random_state=42 for reproducibility.

$$\hat{y} = \sum_{t=1}^{T} f_t(x), \qquad f_t \in \mathcal{F}$$

where f_t is the t-th regression tree added to minimize the regularized objective.

2.5.1.3 *CatBoost*

CatBoost (CB) is a gradient boosting algorithm that introduces ordered boosting to eliminate prediction shift (a form of target leakage that arises in standard gradient boosting when the same samples are used to both compute gradients and build the model) by constructing symmetric (oblivious) trees using random permutations of the training data to derive unbiased leaf value estimates[13]. Imported from the catboost library, it was configured with verbose=0 to suppress all training output and random_state=42 for reproducibility, with the number of iterations left at the library default.

$$\hat{y} = \sum_{t=1}^{T} \eta \, f_t(x)$$

where f_t is a symmetric (oblivious) tree fitted using ordered boosting with random permutations of the training data to eliminate prediction shift, and η is the learning rate.

2.5.1.4 *LightGBM*

LightGBM (LGBM) is a gradient boosting framework that grows trees leaf-wise rather than level-wise, selecting at each step the leaf with the highest potential loss reduction, which produces deeper and more asymmetric trees that converge faster than level-wise methods on large datasets; it additionally employs Gradient-based One-Side Sampling (GOSS) to reduce the number of data instances evaluated at each iteration while retaining samples with large gradients [14]. Imported from the lightgbm library, it was instantiated with random_state=42.

$$\hat{y} = \sum_{t=1}^{T} \eta \, f_t(x)$$

where f_t is a leaf-wise grown tree selected at each step by maximum loss reduction, trained using Gradient-based One-Side Sampling (GOSS) to reduce computation while retaining high-gradient instances.

2.5.1.5 *Gradient Boosting*

Gradient Boosting (GB) is the scikit-learn implementation, which fits additive models in a forward stage-wise manner by minimizing a differentiable loss function using exact greedy split finding and level-wise tree growth. Unlike the other four base learners, GB is sensitive to feature scale in certain configurations, making StandardScaler normalization applied especially relevant to its training stability, whereas the remaining tree-based methods are invariant to monotonic feature transformations [15]. It was configured with n_estimators=100 and random_state=42 for reproducibility.

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + \eta \, f_t(x)$$

where f_t is a regression tree fitted to the negative gradient of the loss function L at iteration t, and η is the learning rate, with T=100 total estimators.

2.5.2 *Adaptive Soft Voting*

The voting mechanism in PhishVote is an adaptive soft vote with rank-based weights, contrasting with the unweighted hard vote used by Saeed [1]. In hard voting, each classifier casts a single vote for the majority class and all votes are treated equally regardless of individual classifier performance. PhishVote replaces this with a soft voting scheme where each base model outputs a full probability vector [P(legitimate), P(phishing)] for a given input, allowing the ensemble to leverage the confidence of each prediction rather than just its direction.

Model weights are determined by ranking all five base models according to their 5-fold stratified cross-validation AUC scores computed on the training set. Ranks are converted to a normalized weight vector w such that all weights sum to 1. The ensemble prediction is:

$$\text{PhishVote}(x) = \underset{c}{\text{argmax}} \left[\sum_{i} w_i \cdot P_i(c \mid x) \right]$$

where w_i is the normalized weight of model i and $P_i(c \mid x)$ is the probability of class c predicted by model i for input x. This formulation ensures that more consistently accurate base models contribute proportionally more to the final decision while preserving the benefit of multi-model diversity.

2.6 Evaluation

PhishVote was evaluated on the held-out 20% test set using standard binary classification metrics. Following the reporting format established in phishing detection literature, results are presented through three complementary formats to enable comprehensive performance assessment and direct comparison with published benchmarks.

First, confusion matrices reporting raw true positive (TP), true negative (TN), false positive (FP), and false negative (FN) counts were generated for the PhishVote ensemble. Second, a per-class classification report was produced, reporting precision, recall, F1-score, and support for both legitimate (class 0) and phishing (class 1) websites, along with overall accuracy and macro and weighted averages. Third, a summary accuracy table was compiled listing accuracy, precision, recall, and F1-score for PhishVote alongside the published results of the Saeed [1] baseline ensemble. Table 4 defines each metric used in the evaluation.

Table 4. Evaluation Metrics

Metric	Formula	Interpretation
Accuracy	*(TP + TN) / (TP + TN + FP + FN)*	Overall proportion of correctly classified URLs
Precision	*TP / (TP + FP)*	Of all URLs flagged phishing, the fraction that are truly phishing
Recall	*TP / (TP + FN)*	Of all actual phishing URLs, the fraction correctly detected
F1-Score	*2 × (P × R) / (P + R)*	Harmonic mean of precision and recall

All experiments used random_state=42 for reproducibility and were implemented in Python using scikit-learn, XGBoost, CatBoost, and LightGBM. The results enable direct comparison between PhishVote and the published Saeed [1] baseline on the UCI Phishing Websites dataset, providing a clear assessment of whether the proposed adaptive soft-voting ensemble with tree-based classifiers achieves measurable performance improvements over the hard voting approach.

3. RESULTS & DISCUSSION

This section presents the experimental results of PhishVote on the UCI Phishing Websites dataset [5] and their comparison against the published Saeed [1] baseline. Section 3.1 presents the EDA findings, Section 3.2 summarizes the Saeed [1] baseline performance, Section 3.3 presents PhishVote's classification results, Section 3.4 provides a direct comparative analysis, and Section 3.5 discusses the findings. All experiments used random_state=42 for full reproducibility.

3.1 Data Analysis

3.1.1 Class Distribution

The class distribution analysis of the UCI-2015 dataset revealed a moderate imbalance, with 6,157 phishing instances (55.7%) and 4,897 legitimate instances (44.3%), yielding an imbalance ratio of 0.795. While this disparity is not severe, it is sufficient to bias a classifier toward the majority class during training, inflating overall accuracy while suppressing recall on the minority class. Synthetic Minority Oversampling Technique (SMOTE) was applied to the training split to synthetically generate minority-class samples by interpolating between existing legitimate instances in feature space, balancing the class distribution prior to model fitting without duplicating existing observations or touching the held-out test set.

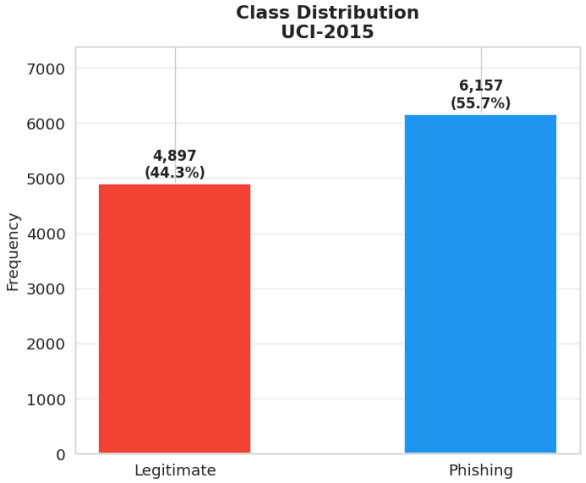

Figure 4. Class Distribution in Phishing Data

3.1.2 Missing Value Audit

A complete audit of all 30 features across 11,054 rows confirmed zero missing values, meaning no imputation was required prior to model training. This is consistent with the dataset's original design, where all features are encoded as deterministic ternary values derived from heuristic rules, leaving no ambiguity in the feature representation.

3.1.3 Feature Distribution

The feature value distribution plots across all 30 UCI-2015 features confirmed that the dataset encodes every feature as a ternary categorical variable taking values in {−1, 0, 1}, where −1 signals a phishing indicator, 0 indicates a neutral or indeterminate state, and +1 indicates a legitimate signal. Importantly, the distributions revealed meaningful class-conditional separation for several high-importance features (HTTPS, AnchorURL, and SubDomains in particular showed starkly different value profiles between phishing and legitimate samples) while features such as DisableRightClick, UsingPopupWindow, and StatusBarCust were heavily skewed toward +1 for both classes, indicating low marginal discriminative power when examined in isolation. Because the Gradient Boosting model in the pipeline is sensitive to feature scale in certain configurations, StandardScaler was applied selectively to that model's input pipeline, while the remaining tree-based classifiers received unscaled features, given their invariance to monotonic feature transformations.

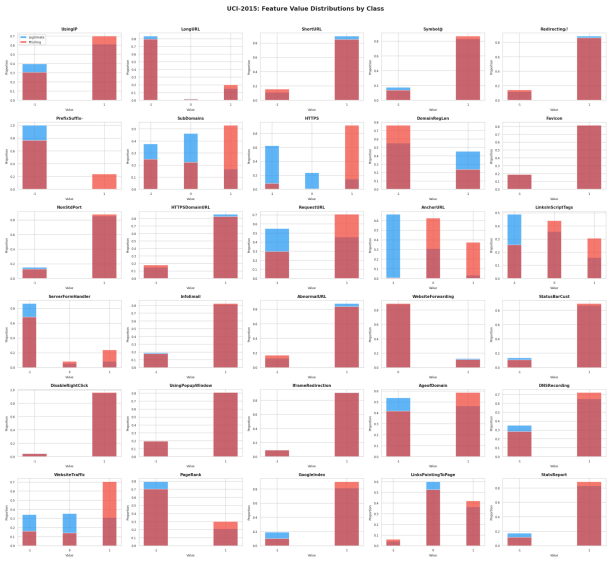

Figure 5. Feature Distribution in Phishing Data

3.1.4 Correlation Heatmap

Two Pearson correlation heatmaps were constructed: one including the target label and one restricted to feature-to-feature correlations. The feature-feature heatmap revealed moderate inter-feature correlations, most notably among StatusBarCust, DisableRightClick, UsingPopupWindow, and IframeRedirection, but no pair exceeded a threshold warranting feature removal. Since all five PhishVote base learners are tree-based and inherently robust to correlated inputs, the full 30-feature set was retained. The feature-to-class heatmap further confirmed meaningful directional correlations with the target label, reinforcing this decision.

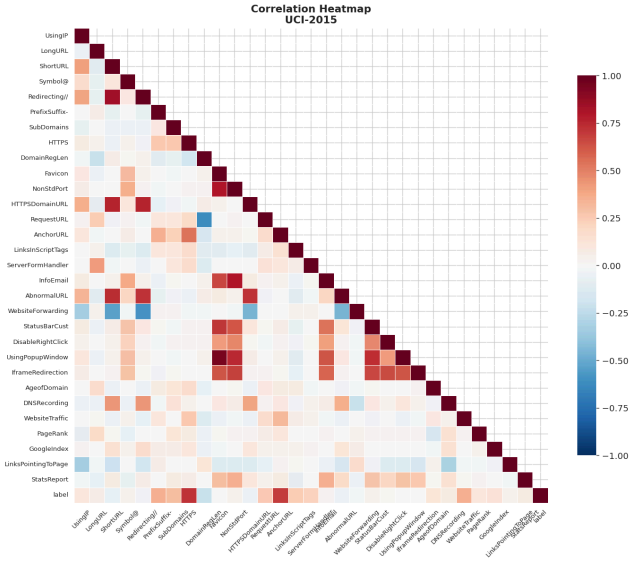

Figure 6. Correlation Heatmap of Phishing Data

3.1.5 Target Correlation Rank

The target correlation rank analysis sorted all 30 features by their absolute Pearson correlation with the binary class label in descending order, producing a ranked bar chart that identified the most discriminating features in the dataset. HTTPS, AnchorURL, and WebsiteTraffic emerged as the top-ranked features by target correlation, consistent with the RF feature importance scores computed post-training. Critically, even the lowest-ranked features (including DisableRightClick, StatsReport, and UsingPopupWindow) retained non-negligible correlation magnitudes and exhibited interpretable class-conditional distributions in the feature distribution plots, indicating that no feature was entirely uninformative. As a result, no dimensionality reduction was applied, and the study proceeded with all 30 features, consistent with the original UCI-2015 feature specification used by Saeed [1] to preserve full comparability.

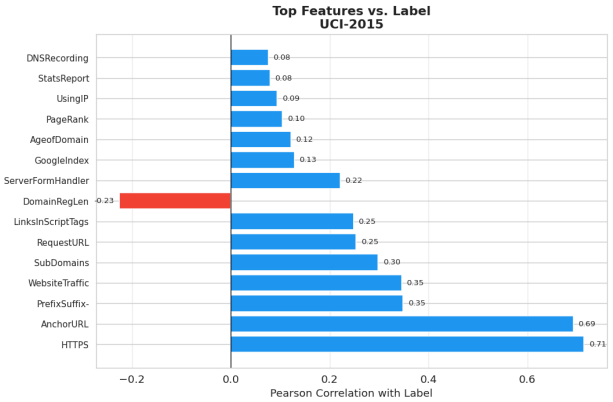

Figure 7. Feature Label Correlation of Phishing Data

3.2 Saeed [1] Baseline Performance

Saeed [1] evaluated a Hard Voting ensemble combining Logistic Regression, Gradient Boosting, and K-Nearest Neighbors on the UCI Phishing Websites dataset, tested on 2,211 instances. The results are presented through confusion matrices and per-class classification reports for each individual model and the Hard Voting ensemble as reported in their study. Table 5 summarizes the per-class performance across all models.

Logistic Regression achieved an accuracy of 93.31%. Its confusion matrix recorded 886 True Negatives and 1,177 True Positives, with 90 False Positives and 58 False Negatives, the highest error counts among the three individual classifiers.

Gradient Boosting achieved an accuracy of 94.93%, recording 911 True Negatives and 1,188 True Positives, with 65 False Positives and 47 False Negatives, the fewest combined errors among the three individual models, indicating the strongest individual generalization performance.

K-Nearest Neighbors achieved an accuracy of 94.35%, with 909 True Negatives and 1,177 True Positives, 67 False Positives and 58 False Negatives, performance closely comparable to Gradient Boosting but with a higher False Negative count.

Hard Voting Ensemble achieved the highest accuracy at 95.02%, with 906 True Negatives and 1,195 True Positives, 70 False Positives and 40 False Negatives. Notably, the ensemble reduced False Negatives to 40, the lowest across all models, indicating stronger phishing detection recall at the cost of a marginal increase in False Positives relative to individual classifiers. This tradeoff reflects the ensemble's tendency to resolve classifier disagreements in favor of the phishing class, prioritizing detection sensitivity over false alarm reduction.

The pattern observed across the baseline study is consistent with a known limitation of hard voting: all base classifiers are treated as equally reliable regardless of individual performance. Logistic Regression at 93.31% carries equal weight to Gradient Boosting at 94.93%, potentially diluting the stronger learner's contribution. A misclassification shared by any two of the three classifiers will override the correct prediction of the third, constraining the ensemble's ceiling performance. This structural constraint motivates the shift toward adaptive soft voting, where model contributions are weighted proportionally to their cross-validated performance rather than treated as interchangeable.

Across all individual models, phishing website recall (class 1) was consistently at or above 0.95, while legitimate website recall (class 0) ranged between 0.91 and 0.93, suggesting that all three classifiers were more sensitive to detecting phishing instances than to correctly clearing legitimate ones. The Hard Voting Ensemble addressed this asymmetry only partially, improving phishing recall to 0.97 while legitimate recall remained at 0.93.

These published results collectively establish the Saeed [1] Hard Voting Ensemble as the reference benchmark for this study. While the baseline demonstrates the viability of ensemble-based phishing detection using classical classifiers, its reliance on uniform voting weights and heterogeneous models of varying accuracy leaves measurable performance headroom that PhishVote is designed to address. The following sections present PhishVote's classification results and a direct comparison against this benchmark.

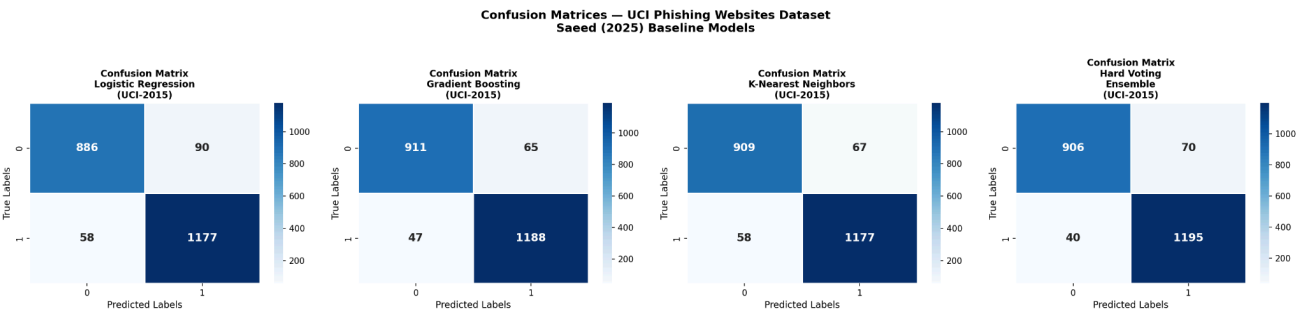

Figure 8. Confusion Matrices of Saeed [1] Baseline Models

Table 5. Saeed [1] Published Classification Results

Model	Accuracy	Class 0 Precision	Class 0 Recall	Class 0 F1	Class 1 Precision	Class 1 Recall	Class 1 F1
Logistic Regression	93.31%	0.94	0.91	0.92	0.93	0.95	0.94
Gradient Boosting	94.93%	0.95	0.93	0.94	0.95	0.96	0.95
K-Nearest Neighbors	94.35%	0.94	0.93	0.94	0.95	0.95	0.95
Hard Voting Ensemble	**95.02%**	**0.96**	**0.93**	**0.94**	**0.94**	**0.97**	**0.96**

These published results serve as the performance benchmark against which PhishVote is evaluated in Section 3.4.

3.3 PhishVote Classification Performance

PhishVote was evaluated on the held-out 20% test set of the UCI Phishing Websites dataset, comprising 2,211 instances: 979 legitimate (class 0) and 1,232 phishing (class 1). The training set class imbalance ratio was 0.795, below the 0.80 threshold, triggering SMOTE application. Each base learner was trained independently before being combined through the adaptive soft-voting mechanism. All models used random_state=42 to ensure full reproducibility across training runs. Results are presented through confusion matrices (Figure 9) and per-class classification reports for each base model and the PhishVote ensemble. Table 6 summarizes the classification performance across all models.

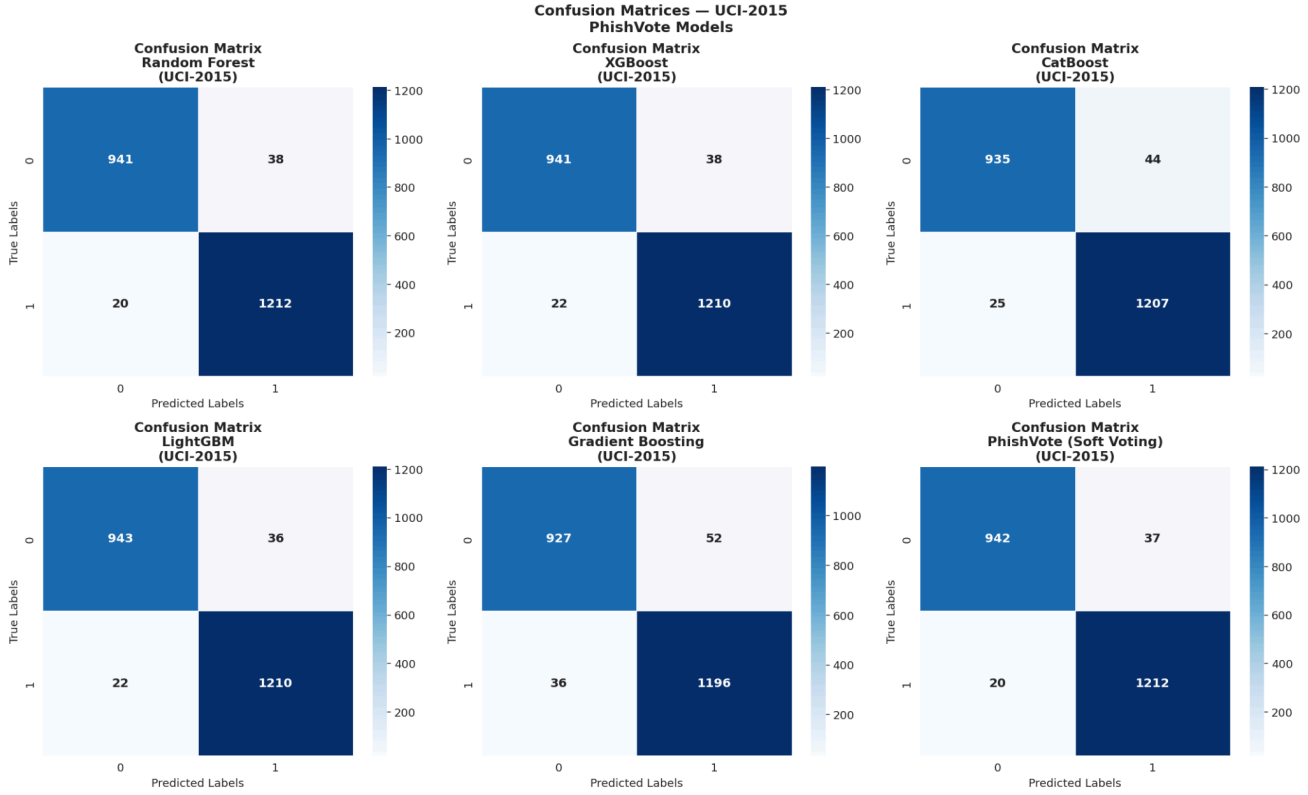

Figure 9. Confusion Matrices of PhishVote Models (UCI-2015)

Table 6. PhishVote Classification Results

Model	Accuracy	Class 0 Precision	Class 0 Recall	Class 0 F1	Class 1 Precision	Class 1 Recall	Class 1 F1
Random Forest	97.38%	0.98	0.96	0.97	0.97	0.98	0.98
XGBoost	97.29%	0.98	0.96	0.97	0.97	0.98	0.98
CatBoost	96.88%	0.97	0.96	0.96	0.96	0.98	0.97
LightGBM	97.38%	0.98	0.96	0.97	0.97	0.98	0.98
Gradient Boosting	96.02%	0.96	0.95	0.95	0.96	0.97	0.96
PhishVote (Soft Voting)	**97.42%**	**0.98**	**0.96**	**0.97**	**0.97**	**0.98**	**0.98**

Random Forest achieved an accuracy of 97.38%, correctly classifying 941 True Negatives and 1,212 True Positives, with 38 False Positives and 20 False Negatives. Precision, recall, and F1-score were 0.98, 0.96, and 0.97 for class 0, and 0.97, 0.98, and 0.98 for class 1, with macro and weighted averages both at 0.97.

XGBoost achieved an accuracy of 97.29%, recording 941 True Negatives and 1,210 True Positives, with 38 False Positives and 22 False Negatives. Precision, recall, and F1-score were 0.98, 0.96, and 0.97 for class 0, and 0.97, 0.98, and 0.98 for class 1, with macro and weighted averages both at 0.97.

CatBoost achieved an accuracy of 96.88%, recording 935 True Negatives and 1,207 True Positives, with 44 False Positives and 25 False Negatives, the highest False Positive count among the five base learners. Precision, recall, and F1-score were 0.97, 0.96, and 0.96 for class 0, and 0.96, 0.98, and 0.97 for class 1, with macro and weighted averages both at 0.97.

LightGBM achieved an accuracy of 97.38%, recording 943 True Negatives and 1,210 True Positives, with 36 False Positives and 22 False Negatives, the lowest False Positive count among all five base learners, indicating the strongest precision on legitimate website classification. Precision, recall, and F1-score were 0.98, 0.96, and 0.97 for class 0, and 0.97, 0.98, and 0.98 for class 1, with macro and weighted averages both at 0.97.

Gradient Boosting achieved the lowest accuracy among the five base learners at 96.02%, recording 927 True Negatives and 1,196 True Positives, with 52 False Positives and 36 False Negatives, the highest combined error count among all base models. Precision, recall, and F1-score were 0.96, 0.95, and 0.95 for class 0, and 0.96, 0.97, and 0.96 for class 1, with macro and weighted averages both at 0.96.

PhishVote (Soft Voting) achieved the highest accuracy among all models at 97.42%, recording 942 True Negatives and 1,212 True Positives, with 37 False Positives and 20 False Negatives, the lowest False Negative count alongside Random Forest, indicating the strongest phishing detection recall. Precision, recall, and F1-score were 0.98, 0.96, and 0.97 for class 0, and 0.97, 0.98, and 0.98 for class 1, with macro and weighted averages both at 0.97. The ensemble's rank-based CV-AUC weighting assigned the highest contribution to XGBoost (0.3333), followed by LightGBM (0.2667), CatBoost (0.2000), Random Forest (0.1333), and Gradient Boosting (0.0667), reflecting the relative cross-validated performance ordering of the five base learners on the training partition.

3.4 Comparative Analysis

Table 7 presents a direct comparison of PhishVote against the published results of the Saeed [1] baseline ensemble on the UCI Phishing Websites dataset across all four evaluation metrics. The comparison spans all ten models evaluated.

Table 7. PhishVote vs. Saeed [1] Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score
Saeed [1] — Logistic Regression	93.31%	0.93	0.95	0.94
Saeed [1] — Gradient Boosting	94.93%	0.95	0.96	0.95
Saeed [1] — K-Nearest Neighbors	94.35%	0.95	0.95	0.95
Saeed [1] — Hard Voting Ensemble	**95.02%**	**0.94**	**0.97**	**0.96**
PhishVote — Random Forest	97.38%	0.97	0.98	0.98
PhishVote — XGBoost	97.29%	0.97	0.98	0.98
PhishVote — CatBoost	96.88%	0.96	0.98	0.97
PhishVote — LightGBM	97.38%	0.97	0.98	0.98
PhishVote — Gradient Boosting	96.02%	0.96	0.97	0.96
PhishVote (Soft Voting)	**97.42%**	**0.97**	**0.98**	**0.98**

The results demonstrate that PhishVote outperforms the Saeed [1] Hard Voting Ensemble across all four evaluation metrics, confirming H_1. PhishVote achieved an accuracy of 97.42% against the baseline's 95.02%, an improvement of 2.40 percentage points. Precision improved from 0.94 to 0.97, recall from 0.97 to 0.98, and F1-score from 0.96 to 0.98 on phishing website classification. All four metrics improved, confirming consistent and comprehensive outperformance.

Notably, every individual PhishVote base learner outperformed the Saeed [1] Hard Voting Ensemble in accuracy, with even the weakest base model, Gradient Boosting at 96.02%, matching the ensemble's F1-score of 0.96. This underscores the performance gap between the two approaches and suggests that the upgrade from classical linear and instance-based classifiers to gradient-boosted tree models.

The adaptive soft-voting ensemble further consolidated these gains, achieving the highest accuracy and F1-score among all ten models evaluated. The rank-based CV-AUC weighting ensured that the strongest base learners, XGBoost and LightGBM, contributed proportionally more to the final prediction, while the inclusion of Gradient Boosting at a low weight (0.0667) preserved ensemble diversity without allowing its comparatively weaker performance to drag down the aggregate result.

On legitimate website classification (class 0), PhishVote improved precision from 0.96 to 0.98 relative to the Saeed baseline, while recall remained comparable at 0.96 versus 0.93 , indicating that PhishVote not only detects more phishing websites correctly but also reduces false alarms on legitimate sites. This dual improvement is practically significant in real-world deployment, where both missed phishing pages and incorrectly flagged legitimate sites carry meaningful user-facing costs.

3.5 Discussion

The experimental results presented in Sections 3.3 and 3.4 demonstrate that PhishVote consistently and substantially outperforms the Saeed [1] Hard Voting baseline across all evaluation metrics on the UCI Phishing Websites dataset. This section interprets these findings in the context of the research objectives, the design decisions underlying PhishVote, and the practical implications for phishing website detection.

Role of Base Learner Upgrade. The most significant contributor to PhishVote's performance improvement is the replacement of Saeed's [1] classical classifiers, Logistic Regression, Gradient Boosting (sklearn defaults), and K-Nearest Neighbors, with five tree-based models trained with higher capacity configurations. The fact that even PhishVote's weakest individual base learner, Gradient Boosting at 96.02%, equals the Saeed [1] ensemble's F1-score suggests that the performance gain is primarily attributable to the upgraded classifier architecture rather than the voting mechanism alone. Gradient-boosted tree models such as XGBoost [12], LightGBM [14], and CatBoost [13] are well-established as superior performers on tabular classification tasks, and the results on the UCI dataset confirm this advantage in the phishing detection domain.

Role of Adaptive Soft Voting. While the base learner upgrade accounts for the bulk of the improvement, the adaptive soft-voting mechanism provides an additional marginal gain over the best individual base learner. PhishVote's ensemble accuracy of 97.42% exceeds the top individual models, Random Forest and LightGBM, both at 97.38%, by 0.04 percentage points. More meaningfully, the ensemble reduced False Negatives to 20, matching Random Forest's count while simultaneously achieving a lower False Positive count of 37 compared to Random Forest's 38. This suggests that the rank-based CV-AUC weighting successfully leveraged the complementary error patterns of the five base learners, producing a more balanced and robust final prediction than any single model could achieve independently [3].

Class-Level Performance. Across both the baseline and PhishVote models, phishing website recall (class 1) consistently exceeded legitimate website recall (class 0). This asymmetry is expected given the class distribution of the UCI dataset, 6,157 phishing instances versus 4,897 legitimate instances, and reflects the models' tendency to optimize toward the more frequent class. PhishVote maintained this pattern while improving legitimate website recall from 0.93 (Saeed baseline) to 0.96, reducing false alarms on legitimate sites. In practical cybersecurity terms, this improvement is meaningful: a system that flags fewer legitimate websites as phishing reduces user friction and erosion of trust in the detection system, while maintaining strong phishing detection sensitivity.

Implications for Ensemble-Based Phishing Detection. The results reinforce the value of employing diverse, high-capacity gradient-boosted tree classifiers as base learners in ensemble phishing detection systems, as advocated by Muhammad et al. [6] and consistent with broader findings in tabular ML benchmarking literature [12], [13], [14], [15]. The adaptive weighting mechanism further demonstrates that not all classifiers contribute equally to ensemble performance, and that allocating proportionally higher weight to more consistently accurate base learners, as measured by cross-validated AUC, produces a more discriminative ensemble than uniform hard voting.

Limitations. Several limitations should be noted when interpreting these results. First, the evaluation is conducted on a static dataset collected in 2015; while the UCI Phishing Websites dataset is a widely used benchmark, phishing techniques have evolved substantially since its collection, and performance on contemporary phishing campaigns may differ. Second, PhishVote relies exclusively on the 30 predefined heuristic features available in the UCI dataset and does not incorporate real-time signals such as DNS resolution, SSL certificate metadata, or WHOIS registration data, which are used in more comprehensive detection frameworks. These limitations define the boundaries of the current study and motivate directions for future work, including evaluation on more recent phishing datasets, integration of richer feature sets, and investigation of lightweight ensemble configurations suitable for real-time deployment.

4. CONCLUSION

This study presented PhishVote, an adaptive soft-voting ensemble classifier for phishing website detection, integrating five tree-based models (Random Forest, XGBoost, CatBoost, LightGBM, and Gradient Boosting) with rank-based cross-validated AUC weights governing each base learner's contribution to the final prediction. Evaluated on the held-out 20% test partition of the UCI Phishing Websites dataset, PhishVote achieved 97.42% accuracy, 0.97 precision, 0.98 recall, and an F1-score of 0.98, surpassing the Saeed [1] Hard Voting Ensemble across all four metrics and confirming the study's Performance Hypothesis. Every individual PhishVote base learner outperformed the Saeed [1] ensemble in accuracy, indicating that the primary driver of improvement is the architectural upgrade to high-capacity gradient-boosted tree models, with the adaptive soft-voting mechanism providing a further marginal but consistent gain through complementary error aggregation.

The primary limitation of this study is that PhishVote is evaluated on a static dataset collected in 2015 and does not incorporate real-time signals such as live DNS resolution, SSL certificate metadata, or WHOIS registration data; performance on contemporary phishing campaigns may therefore differ from reported benchmark metrics. Future work should evaluate PhishVote on more recently collected phishing datasets, investigate real-time browser-integrated deployment configurations, and explore richer dynamically extracted feature sets beyond the static UCI heuristics.

5. REFERENCES

[1] E. M. H. Saeed, "An ensemble voting classifier based on machine learning models for phishing detection," Int. J. Sci. Res. Sci. Eng. Technol. (IJRSRET), vol. 12, no. 1, pp. 15-27, 2025, doi: 10.32628/IJRSRET251211.

[2] Anti-Phishing Working Group (APWG), "Phishing Activity Trends Report," APWG, 2024. [Online]. Available: https://apwg.org/trendsreports/. [Accessed: Feb. 2025].

[3] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL detection using machine learning: A survey," arXiv preprint arXiv:1701.07179, 2017.

[4] T. G. Dietterich, "Ensemble methods in machine learning," in Proc. Int. Workshop on Multiple Classifier Systems, Lecture Notes in Computer Science, vol. 1857, Springer, Berlin, Heidelberg, 2000, pp. 1-15.

[5] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Phishing websites features," UCI Machine Learning Repository, 2015. [Online]. Available: https://archive.ics.uci.edu/dataset/327/phishing+websites. [Accessed: Jan. 2025].

[6] W. Muhammad, S. M. Bhatti, and A. Shaikh, "A comparative study of machine learning algorithms for phishing website detection," Int. J. Adv. Comput. Sci. Appl. (IJACSA), vol. 14, no. 3, pp. 423-432, 2023.

[7] G. Vrbančič, I. Fister Jr., and V. Podgorelec, "Datasets for phishing websites detection," Data in Brief, vol. 33, p. 106438, 2020, doi: 10.1016/j.dib.2020.106438.

[8] N. Abdelhamid, A. Ayesh, and F. Thabtah, "Phishing detection based on associative classification," Expert Syst. Appl., vol. 41, no. 13, pp. 5948-5959, 2014, doi: 10.1016/j.eswa.2014.03.019.

[9] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based machine learning framework," Neural Comput. Appl., vol. 31, no. 8, pp. 3851-3873, 2019, doi: 10.1007/s00521-017-3305-0.

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[11] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), San Francisco, CA, USA, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[13] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Advances in Neural Information Processing Systems (NeurIPS), vol. 31, 2018.

[14] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[15] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, doi: 10.1214/aos/1013203451. [Online]. Available: https://doi.org/10.1214/aos/1013203451.